

Mail2Quote — Autonomous Commercial Insurance Underwriting on Databricks

Executive Summary

Mail2Quote is an end-to-end demo that turns an inbound commercial-insurance quote-request email into a fully priced, PDF-ready quote — with **no human in the loop for the clear-cut cases**, and a clean **underwriter review workflow** for everything in between. It is built for the fictional carrier **BricksHouse Insurance Company**.

A prospective business emails BricksHouse asking for coverage. The email lands in an inbox (a Unity Catalog Volume). Within seconds, Databricks:

1. **Reads** the raw email as soon as it arrives.
2. **Parses** it with a foundation model (Claude) into ~30 structured underwriting fields.
3. **Enriches** it against reference data (existing customers, industry benchmarks, claims/premium norms).
4. **Featurizes** it into ML-ready inputs.
5. **Risk-scores** it with a real-time ML serving endpoint (with a heuristic fallback).
6. **Decides**: auto-approved, auto-declined, or pending-review — with an LLM-written rationale.
7. **Prices** every coverage line (GL, property, WC, auto, cyber, umbrella, etc.).
8. **Generates a PDF quote** and writes it to a Volume.
9. **Closes the loop** for every request in a terminal state table.
10. **Drafts the customer response email** (approval or polite decline).

Cases tagged pending-review surface in a **Databricks App** where a human underwriter can inspect every pipeline stage, ask an LLM questions about the submission, and **approve / decline / request-more-info** — with an optional surcharge or discount. That decision flows *back* into the streaming pipeline automatically, re-pricing and issuing the quote.

The whole thing is a live illustration of the **Data Intelligence Platform**: streaming ingestion, GenAI, classic ML, an operational transactional database, and a production web app — all governed by Unity Catalog in one workspace.

The Business Problem It Solves

Commercial insurance intake is slow and manual. A submission arrives as unstructured text, and a human has to read it, re-key the data, look up the account, judge the risk, price the coverages, write the quote, and email it back. That is hours-to-days of turnaround and expensive underwriter time spent on submissions that are, in many cases, obvious approvals or obvious declines.

Mail2Quote shows how the Databricks platform can:

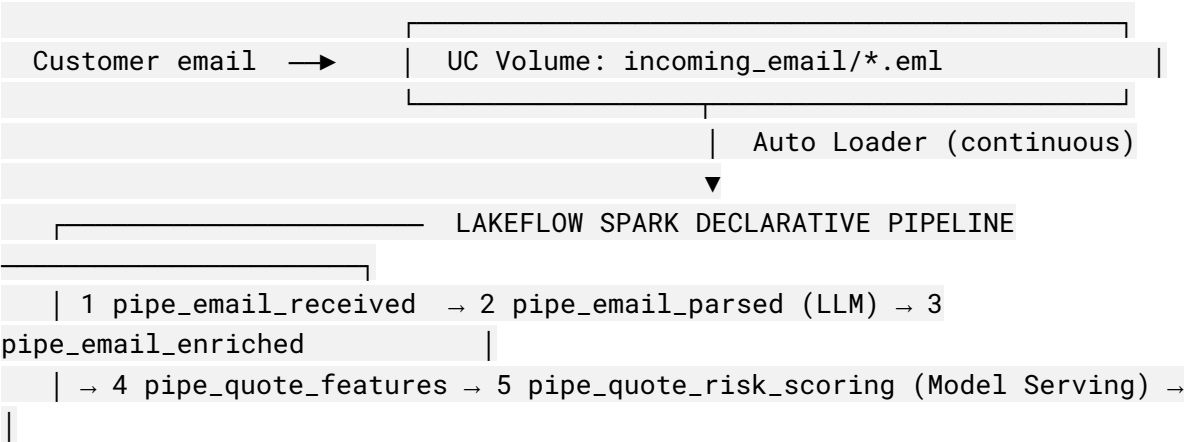
- **Straight-through-process** the clear cases (low-risk → auto-quote, very-high-risk → auto-decline) in seconds.
 - **Route the ambiguous middle** to underwriters with all the context pre-assembled.
 - Keep a **complete, governed audit trail** of every decision and every intermediate artifact in Unity Catalog.
-

Components Used (Databricks Platform)

Component	Role in Mail2Quote
Unity Catalog	Governs the catalog/schema, all 20+ pipeline tables, reference tables, and Volumes. Single source of truth and lineage across the whole flow.
UC Volumes	Three volumes: `incoming_email` (inbox for `.eml` files + the AutoLoader source), `quote_documents` (generated PDF quotes), `outgoing_email` (drafted response `.eml` files).
Auto Loader (`cloudFiles`)	Continuously and incrementally picks up new `.eml` files the instant they land in the inbox volume.
Lakeflow Spark Declarative Pipelines (SDP / DLT)	The heart of the demo — a continuous, streaming pipeline of 10 streaming tables from bronze → silver → gold, plus append flows that inject underwriter decisions back in.
Foundation Model API — `databricks-claude-sonnet-4-5`	Called in-pipeline via `ai_query()` for: (1) email parsing to JSON, (2) the decision rationale summary, (3) the PDF executive summary, and (4) the customer response email. Also called from the app for the underwriter Q&A assistant.
Model Serving	Real-time endpoint `email-to-quote-risk-scorer` scores each submission (claim probability, risk score, loss ratio, pricing/underwriting action). Pipeline falls back to a heuristic if the endpoint is unavailable.
MLflow + AutoML-style training (LightGBM + Optuna)	Trains and registers 3 models — claim classification, risk-score regression, loss-ratio

	regression — tracked in MLflow and registered in the Unity Catalog Model Registry .
Lakebase (managed Postgres / OLTP)	Serves the app with low-latency reads and holds live underwriter decisions. Pipeline (gold) tables are synced to Lakebase for the app to read; the <code>underwriter</code> table is synced back to Delta.
Change Data Feed (CDF)	Powers the reverse sync: underwriter decisions written to Lakebase flow into a CDF-enabled Delta table, which the pipeline's append flows stream from.
Databricks Apps	Hosts the operational web app — a FastAPI backend + React/Vite/TypeScript SPA frontend — running on Databricks with a service principal, connecting to Lakebase and the serving endpoints.
Databricks Jobs	Orchestrates the ML training pipeline and the continuous Lakebase→Delta sync job.
Genie Agent	A natural-language analytics agent over the governed quote data. Underwriters ask portfolio questions in plain English and get instant, SQL-backed answers — benchmarking a submission against the whole book without an analyst or a dashboard.

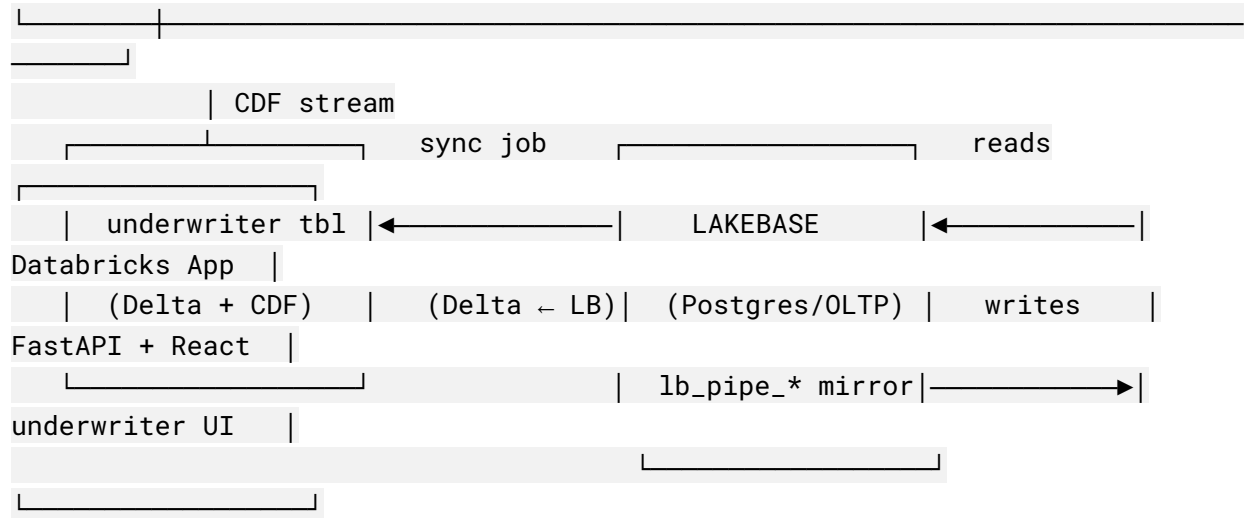
Architecture & Data Flow



```

| 6 pipe_quote_review (decision + LLM summary) → 7 pipe_quote_creation
(pricing) → |
| 8 pipe_pdf_created → 9 pipe_completed → 10 pipe_response_email
|
| ▲
|
| append flows (uw-approved → creation; uw-declined/info →
completed) |

```



Two directions of Lakebase sync are the key architectural trick:

- **UC → Lakebase (for reads):** the gold pipeline tables are mirrored into Lakebase as `lb_pipe_*` tables so the app gets fast, transactional reads without hitting the lakehouse.
- **Lakebase → UC (for writes):** underwriter decisions the app writes to the Lakebase underwriter table are streamed back into a CDF-enabled Delta table (`lb_underwriter_history`). The pipeline's **append flows** read that stream and inject `uw-approved` quotes into pricing and `uw-declined` / `uw-info` quotes into the terminal state — no pipeline restart required.

Design note: the reverse sync uses an **insert-only MERGE** so it doesn't flood the Change Data Feed with spurious `update_postimage` events on every poll — a subtle but important detail for keeping continuous append flows healthy.

The Pipeline Step-by-Step — With Example Output at Each Stage

Below, a single submission from "**Summit Ridge Construction LLC**" (a deliberately higher-risk construction account) is traced through the pipeline. Values are representative of what each table produces.

Input: the incoming email

(incoming_email/quote_request_ab12cd34.eml)

From: john.mercer@summitridgeconstruction.com

To: underwriting@brickshouse-insurance.com

Subject: Commercial Insurance Quote Request - Summit Ridge Construction

Dear Underwriting Team,

We are requesting a commercial insurance quote for Summit Ridge Construction LLC.

Below is a summary of our operations:

Business Details:

- Legal Name: Summit Ridge Construction LLC
- NAICS: 236220
- Established: 2019
- Location: Denver, CO
- Annual Revenue: \$8,500,000
- Annual Payroll: \$3,200,000
- Employees: 42 (plus 15 contractors)
- Uses subcontractors: Yes

Coverage Requested:

1. General Liability: \$2,000,000 limit, \$10,000 deductible
2. Commercial Property: \$1,500,000 limit
3. Workers Compensation: statutory
4. Commercial Auto: \$1,000,000 limit
5. Umbrella / Excess Liability: \$5,000,000 limit

Fleet: 12 vehicles (pickup trucks, flatbeds)

Loss History (recent claims):

- 2023: Ladder fall injury - \$85,000 (closed)
- 2022: Vehicle collision - \$42,000 (closed)
- 2021: Property water damage - \$18,000 (closed)

Current Carrier: Acme Mutual, renewing 2026-06-30
Current Premium: ~\$142,000 annually

Best regards,
John Mercer
Risk Manager, Summit Ridge Construction LLC
(303) 555-0142

Step 1 — pipe_email_received (Bronze, streaming)

Auto Loader captures the raw file and stamps a UUID + file metadata.

email_id	file_name	file_size	ingestion_timestamp
`ab12cd34-...-9f01`	quote_request_ab12cd34.eml	1,184	2026-09-21T14:03:11Z

(raw_content holds the full email text above.)

Step 2 — pipe_email_parsed (Silver, streaming, LLM)

ai_query('databricks-claude-sonnet-4-5', ...) extracts ~30 structured fields. The model returns JSON, which is parsed into typed columns:

```
{
  "sender_name": "John Mercer",
  "sender_email": "john.mercer@summitridgeconstruction.com",
  "business_name": "Summit Ridge Construction LLC",
  "naics_code": "236220",
  "risk_category": "construction",
  "date_established": "2019",
  "annual_revenue": 8500000.0,
  "annual_payroll": 3200000.0,
  "num_employees": 42,
  "coverages_requested": "general_liability, property, workers_comp, commercial_auto, umbrella",
  "gl_limit_requested": 2000000.0,
  "property_tiv": 1500000.0,
  "auto_fleet_size": 12,
  "umbrella_limit_requested": 5000000.0,
  "num_claims_5yr": 3,
  "total_claims_amount": 145000.0,
  "worst_claim_description": "Ladder fall injury",
```

```

"current_carrier": "Acme Mutual",
"current_premium": 142000.0,
"has_safety_procedures": true,
"has_employee_training": true,
"urgency": "standard"
}

```

Step 3 — pipe_email_enriched (Silver, streaming)

Joins the parsed record with reference tables (organizations, claims, policies, locations, financials) to add industry benchmarks and match existing customers.

Field	Value
is_existing_customer	`false` (no email match)
industry_avg_claims_per_org (construction)	2.4
industry_avg_claim_severity	\$61,200
industry_avg_premium	\$118,500
industry_avg_revenue	\$6,900,000
revenue_vs_industry_pct	123% (larger than peers)
premium_vs_industry_pct	120%
claims_vs_industry	+0.6 (above average frequency)

Step 4 — pipe_quote_features (Gold, streaming)

Computes the ML-ready feature vector. A sample of the ~35 features:

Feature	Value
business_age_years	7
payroll_per_employee	\$76,190
num_coverages_requested	5
avg_claim_severity	\$48,333
high_claim_frequency_flag	0 ($3 \leq 3$)

safety_score	2
risk_category_score	1.0 (construction = highest)
heuristic_risk_score	~58 / 100

Step 5 — pipe_quote_risk_scoring (Gold, streaming, **Model Serving**)

The feature vector is POSTed to the email-to-quote-risk-scorer serving endpoint.
Output (or heuristic fallback):

Field	Value
claim_prediction	1
risk_score	62.4 / 100
predicted_loss_ratio	0.71
risk_band	High
pricing_action	loaded_rate
underwriting_action	senior_underwriter_review
scoring_method	`model` (or `heuristic`)

Step 6 — pipe_quote_review (Gold, streaming, **LLM**)

Applies the decision rule and asks the LLM for a rationale.

Decision rule:

- risk_score ≤ 30 and no claim predicted → **auto-approved**
- risk_score > 80 or decline_or_refer → **auto-declined**
- otherwise → **pending-review**

For Summit Ridge (score 62.4) → **`pending-review`**.

Example review_summary (LLM-generated):

"This construction account presents moderate-to-elevated risk. With three claims totaling \$145K over five years — including a \$85K ladder-fall injury — loss frequency runs slightly above the construction-industry benchmark, and the predicted loss ratio of 0.71 approaches the profitability threshold. Written safety procedures and an active training program are mitigating factors, but the subcontractor exposure and fleet of 12 vehicles warrant senior underwriter review before a quote is issued."

(A low-risk professional-services account here would instead be tagged `auto-approved` and flow straight to Step 7; a score >80 account would be `auto-declined` and skip to Step 9.)

Step 7 — pipe_quote_creation (Gold, streaming)

For approved quotes, prices each coverage line using rate × exposure × multipliers (risk multiplier, industry multiplier, experience mod). Example for Summit Ridge once approved by an underwriter:

Coverage Line	Premium
General Liability	\$38,250.00
Property – Building	\$6,825.00
Property – Contents	\$3,600.00
Business Income	\$1,062.50
Equipment Breakdown	\$300.00
Workers Compensation	\$55,200.00
Commercial Auto	\$30,000.00
Umbrella / Excess	\$9,375.00
Terrorism (TRIA)	\$487.00
Policy Fees	\$150.00
Subtotal Premium	\$145,249.50
Quote Number	`QT-20260921-ab12cd34`
Effective / Expiration	2026-10-05 → 2027-10-05

If an underwriter applied, say, a 10% surcharge, `total_premium` becomes ~\$159,774; the app shows both subtotal and adjusted premium.

Step 8 — pipe_pdf_created (Gold, streaming)

Generates a branded PDF quote (via the `pdf_generator` module loaded from the Volume), writes it to `quote_documents/QT-20260921-ab12cd34.pdf`, and records status.

quote_number	pdf_status	pdf_path
QT-20260921-ab12cd34	`generated`	/Volumes/.../quote_documents/QT-20260921-ab12cd34.pdf

The PDF also carries an LLM-written **executive summary** paragraph, e.g.:

"BricksHouse Insurance is pleased to present this commercial package proposal for Summit Ridge Construction LLC, a Denver-based construction firm. The program provides \$145,250 in total annual premium across general liability, property, workers' compensation, commercial auto, and a \$5M umbrella, effective October 5, 2026. Pricing reflects the account's High risk band (62.4/100) and its recent loss experience, balanced against strong safety controls."

Step 9 — pipe_completed (Gold, streaming — terminal state)

Unifies **every** request into a final state, whether it was PDF-issued or declined.

final_status	source	example
`quote_issued`	approved + PDF	Summit Ridge (after UW approval)
`auto_declined`	risk_score > 80	a distressed trucking account
`uw_declined` / `uw_info_requested`	underwriter action	via append flow

Step 10 — pipe_response_email (Gold, streaming, LLM)

Drafts the outbound customer email and writes an .eml to outgoing_email/. Approved example:

Subject: Your Commercial Insurance Quote QT-20260921-ab12cd34 - Summit Ridge Construction LLC

Dear John, Thank you for your commercial insurance quote request for Summit Ridge Construction LLC. We are pleased to provide the following quote... **Total Annual Premium: \$145,249.50**... This quote is valid for 30 days. To bind this policy, reply to this email... — *BricksHouse Insurance Underwriting Team, (555) 123-4567.*

Declined submissions receive a polite, empathetic decline drafted by the same model.

The Underwriter App (Databricks App)

The pending-review cases (like Summit Ridge) are where the human comes in. The app is a **FastAPI + React** application deployed on Databricks Apps, reading/writing Lakebase.

Key screens & capabilities:

- **Quote pipeline view** — every submission with a live progress tracker across all 10 steps (received → parsed → enriched → features → riskscoring → review → creation → pdf → completed → responseemail). Click any step to inspect that table's full row.
- **Underwriter queue** — pending-review quotes, oldest first, with the full parsed submission, risk score, band, predicted loss ratio, pricing action, and the LLM rationale.

- **"Ask about this quote"** — a free-text box that sends all pipeline data for that submission to Claude and returns an underwriting answer (e.g. *"Is the WC exposure driving most of the premium?"*).
- **Decision panel** — the underwriter picks **Approve / Decline / Request Info**, optionally setting a **surcharge %** or **discount %** and notes. On submit:
 - **uw-approved** → append flow re-prices and issues a PDF quote.
 - **uw-declined** → routed to terminal state.
 - **uw-info** → an "additional information required" email is generated to the customer.
- **Email intake / send** — pre-built sample emails can be dropped into the inbox volume to trigger the pipeline live during a demo; response emails can be (re)generated on demand.
- **Analytics dashboard** — totals by decision, auto-approve/decline vs. pending rates, average end-to-end completion time, average underwriter delay, and average premium/risk by industry category.

Example API payload — an underwriter approving with a 10% surcharge:

POST /api/underwriter/decide

```
{
  "email_id": "ab12cd34-...-9f01",
  "decision": "uw-approved",
  "surcharge_pct": 10,
  "discount_pct": 0,
  "notes": "Approved with 10% surcharge given elevated loss frequency; strong
safety controls noted."
}
```

That single write lands in Lakebase, is streamed back to Delta via CDF, and the append flow re-enters the pipeline — issuing QT-20260921-ab12cd34 at the adjusted premium.

Genie — portfolio context at decision time

A pending-review submission is far easier to judge when the underwriter can see how it compares to the existing book of business. A Databricks Genie Space sits on the governed quote data — the same Unity Catalog and Lakebase tables the pipeline produces — and lets underwriters ask questions in plain English, right next to the quote they are reviewing:

- *What is our average premium and loss ratio for construction accounts with three or more claims?*
- *How many similar risks did we auto-decline in the last 90 days, and why?*
- *What did we charge the last five retail accounts of this size?*

Genie translates each question to SQL, runs it against the live data, and returns the answer (and the query it used) in seconds. Instead of waiting on an analyst or paging through a dashboard, the underwriter gets instant benchmarking and precedent — turning an approve / decline / refer / adjust decision that once took days into one made in minutes. Because Genie

runs on Unity Catalog-governed data, every answer respects the same permissions and lineage as the rest of the platform.

The ML Pipeline (Offline Training)

Separate from the streaming flow, an ML pipeline (a multi-notebook Databricks Job) builds the risk models that Step 5 serves:

1. ``01_feature_engineering`` — builds a `risk_features` table from the reference data with train/test split.
2. ``02_automl_training`` — trains **three LightGBM models** with **Optuna** hyperparameter search (20 trials each), tracked in **MLflow**:
 - **Claim classification** (`has_claim`) — reports F1 / precision / recall.
 - **Risk-score regression** (`risk_score`, 0–100) — reports RMSE / MAE / R^2 ; deliberately excludes claim-derived features to avoid leakage.
 - **Loss-ratio regression** (`loss_ratio`) — reports RMSE / MAE / R^2 .
1. ``03_model_registration`` — registers the best runs to the **Unity Catalog Model Registry**.
2. ``04_model_serving`` — deploys the `email-to-quote-risk-scorer` real-time endpoint.
3. ``05_monitoring`` — monitors the endpoint / model quality.

Example training console output:

```
=== Claim Classification - Test Set ===
  F1: 0.8123 | Precision: 0.7940 | Recall: 0.8315
=== Risk Score Regression - Test Set ===
  RMSE: 6.42 | MAE: 4.88 | R2: 0.87
=== Loss Ratio Regression - Test Set ===
  RMSE: 0.14 | MAE: 0.10 | R2: 0.79
```

Why This Is a Compelling Demo

- **Everything in one platform, one workspace** — ingestion, GenAI, ML, OLTP serving, and a production app, all governed by Unity Catalog with end-to-end lineage.
- **GenAI + classic ML together** — the LLM handles the unstructured language work (parsing, rationale, correspondence) while LightGBM handles the quantitative risk scoring. Neither is forced to do the other's job.
- **Real-time, continuous** — drop an email in the volume and watch it flow through all 10 stages in seconds via a continuous SDP pipeline.
- **Human-in-the-loop done right** — the underwriter's decision doesn't live in a silo; it's written to Lakebase and streamed *back into* the same pipeline through CDF append flows.

- **Operational, not just analytical** — Lakebase gives the app transactional reads/writes, so the demo behaves like a real production underwriting system, not a batch report.

Prepared from the Mail2Quote demo source (BricksHouse Insurance / FINS Canada). Example values are representative illustrations of each pipeline stage's output.